```
import numpy as np
from sklearn.datasets import fetch_openml
from sklearn.ensemble import RandomForestClassifier, ExtraTreesClassifier, VotingC
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
from sklearn.linear_model import LogisticRegression
```

```
# 1. Load the MNIST data
# fetch_openml will download the dataset. It might take a minute or two.
print("Loading MNIST dataset...")
# Using parser='auto' is recommended for future compatibility
mnist = fetch_openml('mnist_784', version=1, as_frame=False, parser='auto')
X, y = mnist["data"], mnist["target"]
# The data is loaded as strings, so we convert y to integers
y = y.astype(np.uint8)
print("Dataset loaded.")
```

```
Loading MNIST dataset...
Dataset loaded.
```

```
# 2. Split the data into training, validation, and test sets
# The MNIST dataset is already split into a training set (the first 60,000 instanc
# and a test set (the last 10,000 instances). We'll further split the training set
X_train_val, X_test = X[:60000], X[60000:]
y_train_val, y_test = y[:60000], y[60000:]

# Split the training set into a smaller training set and a validation set
X_train, X_val = X_train_val[:50000], X_train_val[50000:]
y_train, y_val = y_train_val[:50000], y_train_val[50000:]

print(f"Training set size: {len(X_train)}")
print(f"Validation set size: {len(X_val)}")
print(f"Test set size: {len(X_test)}")
```

```
Training set size: 50000
Validation set size: 10000
Test set size: 10000
```

```
# 3. Train various classifiers
# Initialize the classifiers.
# We set probability=True for SVC to enable soft voting. This will slow down train
# n_jobs=-1 uses all available CPU cores.
rnd_clf = RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-1)
ext_clf = ExtraTreesClassifier(n_estimators=100, random_state=42, n_jobs=-1)
svm_clf = SVC(gamma="scale", probability=True, random_state=42)
```

```
# 4. Create and train a soft voting ensemble
# A soft voting classifier often performs better than a hard one.
voting_clf = VotingClassifier(
    estimators=[('rf', rnd_clf), ('et', ext_clf), ('svc', svm_clf)],
    voting='soft',
    n_jobs=-1
)

# Train all the classifiers on the training set
print("\nTraining classifiers...")
classifiers = [rnd_clf, ext_clf, svm_clf, voting_clf]
for clf in classifiers:
    print(f"Training {clf.__class__.__name__}...")
    clf.fit(X_train, y_train)
print("Training complete.")
```

```
Training classifiers...
Training RandomForestClassifier...
Training ExtraTreesClassifier...
Training SVC...
Training VotingClassifier...
Training complete.
```

```
print("\nEvaluating models on the validation set...")
for clf in classifiers:
    y_pred = clf.predict(X_val)
    accuracy = accuracy_score(y_val, y_pred)
    print(f"{clf.__class__.__name__}: {accuracy:.4f}")
```

```
Evaluating models on the validation set...
RandomForestClassifier: 0.9736
ExtraTreesClassifier: 0.9743
SVC: 0.9802
VotingClassifier: 0.9813
```

```
print("\nBuilding stacking ensemble...")

# Generate predictions on validation set
val_preds_rf = rnd_clf.predict(X_val)
val_preds_et = ext_clf.predict(X_val)
val_preds_svm = svm_clf.predict(X_val)
```

```
Building stacking ensemble...
```

```
# New training set for blender
X_blender = np.c_[val_preds_rf, val_preds_et, val_preds_svm]
y_blender = y_val
```

```
# Train blender (Logistic Regression)
blender = LogisticRegression(max_iter=1000, random_state=42)
blender.fit(X_blender, y_blender)
print("Blender training complete.")
```

```
Blender training complete.
```

```
#  Evaluate Stacking Ensemble on Test Set
print("\nEvaluating stacking ensemble on the test set...")

# Test predictions from base models
test_preds_rf = rnd_clf.predict(X_test)
test_preds_et = ext_clf.predict(X_test)
test_preds_svm = svm_clf.predict(X_test)

# Stack into test feature matrix
X_test_blender = np.c_[test_preds_rf, test_preds_et, test_preds_svm]

# Blender makes final predictions
final_preds = blender.predict(X_test_blender)
stacking_accuracy = accuracy_score(y_test, final_preds)
print(f"Stacking Ensemble Test Accuracy: {stacking_accuracy:.4f}")
```

```

Evaluating stacking ensemble on the test set...
Stacking Ensemble Test Accuracy: 0.9684
```

```
# Compare Performance
print("\nEvaluating individual models on the test set...")
results = {}

for clf in (rnd_clf, ext_clf, svm_clf, voting_clf):
    y_pred = clf.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    results[clf.__class__.__name__] = acc
    print(f"{clf.__class__.__name__} Test Accuracy: {acc:.4f}")

results["StackingEnsemble"] = stacking_accuracy

print("\nFinal Comparison of Test Accuracies:")
for model, acc in results.items():
    print(f"{model}: {acc:.4f}")
```

```

Evaluating individual models on the test set...
RandomForestClassifier Test Accuracy: 0.9680
ExtraTreesClassifier Test Accuracy: 0.9703
SVC Test Accuracy: 0.9785
VotingClassifier Test Accuracy: 0.9783
```

```
Final Comparison of Test Accuracies:
RandomForestClassifier: 0.9680
ExtraTreesClassifier: 0.9703
SVC: 0.9785
VotingClassifier: 0.9783
StackingEnsemble: 0.9684
```

```
Start coding or generate with AI.
```